

Case Study: Visme's Content Forge

- **Role Targeted:** Senior Product Designer
- **Timeline:** 8-hour day sprint
- **Toolchain:** Gemini, Claude (Sonnet 4.6 & Opus 4.7), Claude Code, Claude Design

Phase 1: Understanding the Product

Visme is a complex product. More complex than it appears from the outside. It sits in a category similar to Canva, but operates on different logic. Canva optimizes for visual expression, making things look good. Visme optimizes for business communication, making information land. The difference shows up everywhere: in the template library (skewing toward reports, org charts, data visualizations), in the target user (business analysts and marketing managers rather than creatives), and in the editor itself (more structured, less freeform). This distinction matters for AI integration. While Canva's AI should make things more beautiful, Visme's AI should make content more effective.

Phase 2: JTBD Discovery

Before writing code or opening a design tool, I wanted to understand what actual growth and success look like for Visme's enterprise users. Hence, I started by reading through the five featured customer examples on the Visme website. Two caught my attention:

- **WOW! (Telecommunications Learning & Dev):** Highlighted the friction of non-designers jumping into visual layouts, often breaking carefully aligned templates because text copy exceeded geometric boundaries.
- **QSN (Compliance Consultancy):** Emphasized the deep cognitive load placed on non-designers trying to condense heavy, contextual regulatory text into tight visual formats like infographic sidebars.



The Job-To-Be-Done (JTBD)

Situation: A business professional or marketer needs to communicate the same message across different visual surfaces at the same time.

Motivation: They want to ensure that copy naturally fits the constraints of each layout without overflowing containers or losing meaning in truncation.

Expected Outcome: On-brand content shipped fast, without needing a designer or a copywriter to manually adjust text or bounding boxes for every format.

Phase 3: Brainstorming with Human-in-the-Loop AI

With the main problem in place I used Gemini as the primary brainstorming tool. LLM's naturally skew toward expansive feature suites, so I acted as the AI constraint mechanism while defining the scope for the actual app. Besides feasibility, it was necessary to define a solution that can effectively be built in a day.

The idea I landed on was:

Content Forge: The Smart Contextual Content Truncator

An intelligent interface takes raw text input, interprets its structural intent, and automatically maps optimized variants into different layouts.

Source Content

86 words

Paste or edit the canonical text the engine will reformat.

SOURCE TYPE

Financial

Product

Compliance

TEXT • Financial

Q3 2025 Financial Summary

Revenue: \$42.1M (+18% YoY, +6% QoQ)
Gross margin: 73.4%, up 1.8 pts from Q2.
Cash position: \$128M, runway > 30 months.

Drivers

- Enterprise ARR crossed \$24M, ~57% of total.
- Net retention 119%, highest since IPO.
- Marketing-attributed pipeline +34% QoQ.

Format & Distribute

Phase 4: Pre-Build & Reverse-Engineering the Design System

Visme does not have a publicly available design system. As I wanted the prototype to feel like a native extension of their platform ecosystem, I initially turned to automated design system engineering. Which meant extracting a design system from the product itself.

The flow worked as follows.

Extraction Pipeline: I scraped structural DOM files directly from three sources: visme.co marketing site, active dashboard viewports, and creation editor backend. For the scrape I leveraged **design-extract**, an advanced fork of the Dembrandt codebase, to systematically pull colors, padding scales, and typographic scales out of the raw layout code.

What became clear immediately after the scrape was that Visme operates three distinct visual layers, marketing, dashboard, and editor, each with a somewhat different design language. The editor is significantly more neutral and functional than the marketing site, which uses the brand's full color range aggressively.

AI Synthesis: Using **Claude Code**, I synthesized these disparate style scripts into a standardized design system architecture token set. Conflicts between the three layers were resolved by prioritizing dashboard values. The intended editor values could not be used, as the scrape could not retrieve the complete token structure. This custom set was then loaded into **Claude Design** to scaffold the Visme (Quasi) Design System.

Note: This was my first attempt to run an end-to-end design system extraction pipeline. I was preparing to start integrating Claude Design into my workflow, and this was my first opportunity to do so.

Components

- > Components · Avatars •
- > Components · Badges & tags •
- > Components · Buttons (app) •
- > Components · Buttons (marketing) •
- > Components · Cards •
- > Components · Form controls •

Components · Inputs

Default, focus, error, disabled, search

👍 Looks good

🗨 Needs work...

Workspace name

Untitled workspace

Email

hello@visme.co

Focused state

Password

•••

Must be at least 8 characters

Plan

Pro · seat-locked

Search templates

🔍 Search by name, tag, or c

- > Components · Menus & selects •
- > Components · Tabs & nav •

Phase 5: Prototyping & Code Refactoring

With the design engine in place, the build phase was executed in a series of rapid engineering sprints.

The Visual Sprint

Component & Layout Generation (Claude Design): Using the extracted Visme design files, I proceeded with component generation inside Claude Design using the extracted Visme tokens. Subsequently, these components were then used to build the initial layout of the Content Forge app.

Interactive Architecture (Claude Code, Sonnet 4.6): The design was then exported to Claude where I wired up the interactive elements to handle real-time UI state modification (React hooks). As this is a demo, input and output content was hardcoded with Babel.

Refinement & Engineering Craft

Accessibility Matrix: Once the layout states functioned, I systematically added clean visual contrast ratios, accessible focus controls, and aria semantics to ensure the component assembly respected inclusive engineering benchmarks.

Code Optimization (Claude Code, Opus 4.7): I passed the working prototype code base to Claude Opus 4.7 for high-order architectural refactoring. Opus successfully minimized styling logic redundancy, optimized component state lookups, and cleaned up the Babel runtime footprint.

Responsive Fluidity & Dark Mode: As a final layer of platform fidelity, I built out clean CSS breakpoint fluid metrics and integrated a seamless global Dark Mode controller option toggle directly on the dashboard canvas interface.

Your Content

Type or paste any relevant content. Our AI will adapt it automatically to fit each target area automatically. You're viewing a demo. For the purpose of the demo, select a content preset below to see how adaptation works. In the full version, you'll be able to type or paste your own content.

CONTENT EXAMPLES

- Financial Goal
- Product Launch
- Compliance Rule

SOURCE TEXT

Our primary strategic focus for the upcoming third quarter is to aggressively scale our enterprise client acquisition metrics by an estimated 25% year-over-year, while concurrently optimizing our overall operational overhead costs across all internal sub-departments to ensure maximum profitability and lean resource distribution.

43 words

Adapt Content

Adapted Formats

Three formats adapted from your content.

Live

Presentation Slide

16:9 · slide cover

FINANCIAL GOAL

Q3 Strategic Focus

Scaling Enterprise Growth

Copy

Rewrite

Default

Infographic

Portrait 4:5 · long-form visual

Financial Goal

01 Accelerate enterprise acquisition by 25%

02 Optimize operational overhead costs

03 Maximize cross-department profitability

visme.co

Copy

Rewrite

Default

Social Post

1:1 · square post

Visme

We're setting a bold path for Q3. Our primary goal is to scale enterprise acquisition by 25% while maximizing our internal operational efficiency. Leaner, faster, and focused on growth. Let's build!

#EnterpriseGrowth #SaaS #BusinessStrategy

Copy

Rewrite

Default

Key Takeaways & Product Insights

AI as an Infrastructure, not Interface: Content Forge demonstrates that AI inside a content platform should not manifest as a chat drawer bolted onto a canvas. It should operate as an implicit background layer that handles contextual constraints on behalf of the user. The user states their intent once, the system adapts the output for every surface automatically.

Coded prototypes over static artboards: Bypassing Figma to build a live, working React layout in hours proves the value of design-engineering convergence. Layout boundaries, animation micro-interactions, and systemic scale can be evaluated with real production evidence on day one.

What I would build next?

Content Forge is a working demo, not a shipped product. Given more time, I would develop it into a real application with three meaningful upgrades.

First, replace the hardcoded content data with a live Claude API call. The user writes or pastes any raw content and the AI adapts it in real time, not from preset options.

Second, refine the design system extraction pipeline properly. This would be achieved by building a URL-to-DESIGN.md extraction tool ruleset that defines how pages are scraped, interprets the design system semantically, and outputs a structured file ready for Claude Design ingestion.

Third, when given the chance I would wire the adapted outputs directly into Visme's creation editor via API. The Visme user will then get to change content in all layout slots with the correct formatting at a button click.